

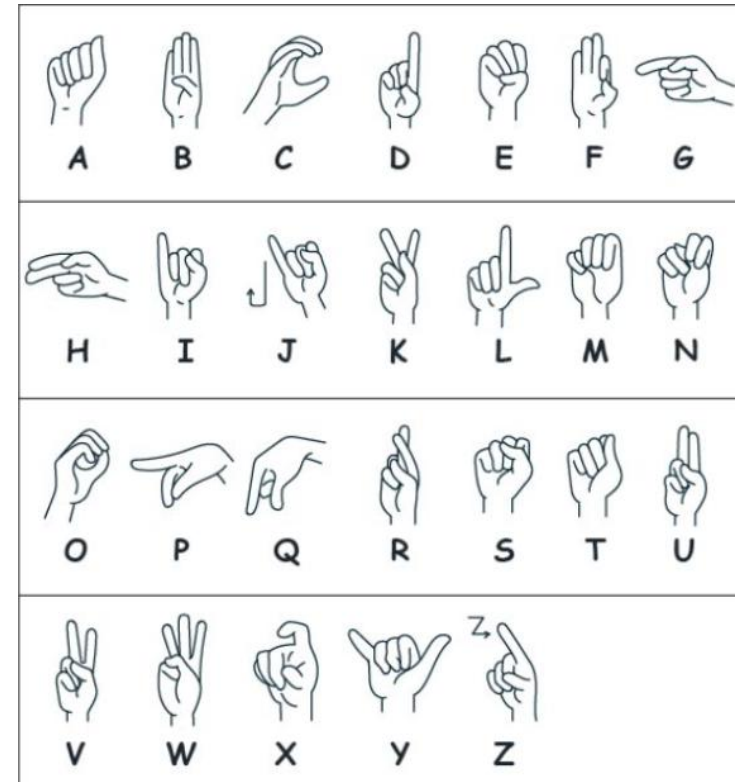
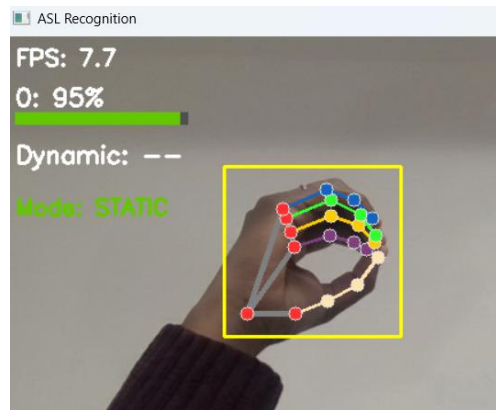
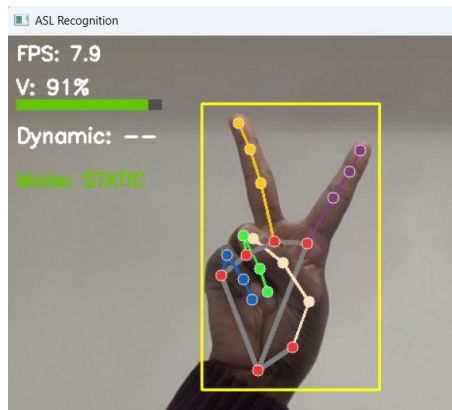
LIGHTWEIGHT TEMPORAL DEEP LEARNING MODEL FOR REAL-TIME AMERICAN SIGN LANGUAGE RECOGNITION

Student: Tresa Joby Thommasseri Antony

Supervisor: Professor, Dr.sc.ing. Aleksandrs Grakovskis

What is ASL Recognition?

Sign language is a crucial communication medium for the hearing and speech impaired. This research proposes the development of a real-time American Sign Language (ASL) recognition system capable of identifying both static and dynamic gestures (some common word such as “hello”, “thank you”, “yes”, “no” etc).



Object of the Study

The object in question is the ASL recognition system as such the software artefact that consist of:

- A hand detector pipeline and landmark extractor pipeline based on MediaPipe.
- Multi-layer perceptron (MLP) classifier of poses on a static hand.
- A bi-directional LSTM (BiLSTM) sequence classifier of dynamic gestures.
- A unified interface that is also a real-time inference engine that combines both the models.

Study Object

The study topic is the gesture recognition in American Sign Language as a human-computer interaction problem.

Specifically:

- Still signs: 24 letters of the ASL alphabet (A-Y, but not J, Z) and numbers 0-9 -> 34 classes in total.
- Dynamic gestures: Letters J and Z and 10 frequent words: hello, thanks, please, sorry, yes, no, help, stop, more, finish → 12 classes in total.
- The experiment is devoted to single-hand gestures recorded with the help of the ordinary webcam in daylight.

Goal of the Research

The main objective of this project is to develop a real-time ASL recognition system. The system will recognize basic static ASL hand signs, such as letters and digits, as well as a few dynamic gestures in real time. The project also aims to achieve at least 90% recognition accuracy and ensure smooth real-time performance on a typical consumer laptop. To protect user privacy, the system will process only hand landmark data instead of storing raw video or images.

Research questions

This study aims to answer the following research questions:

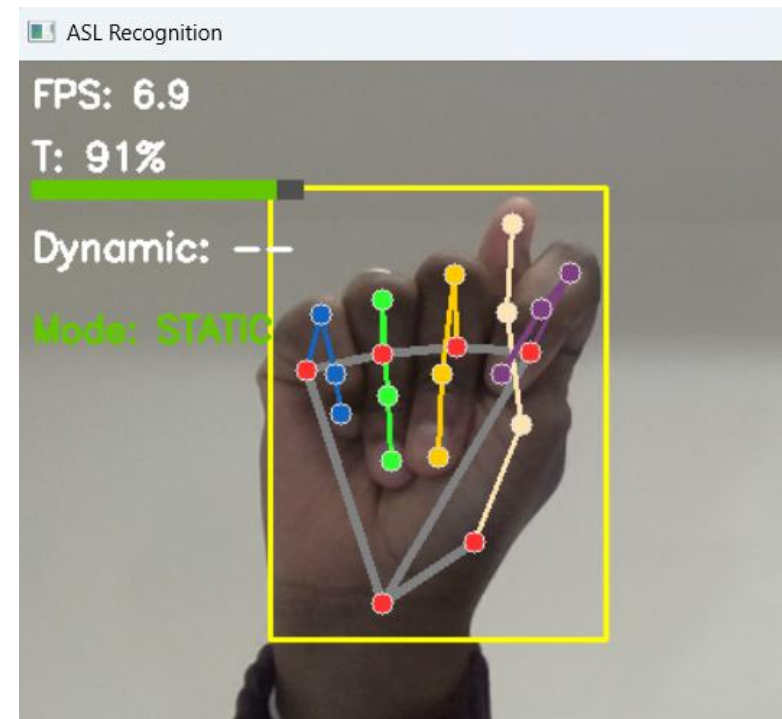
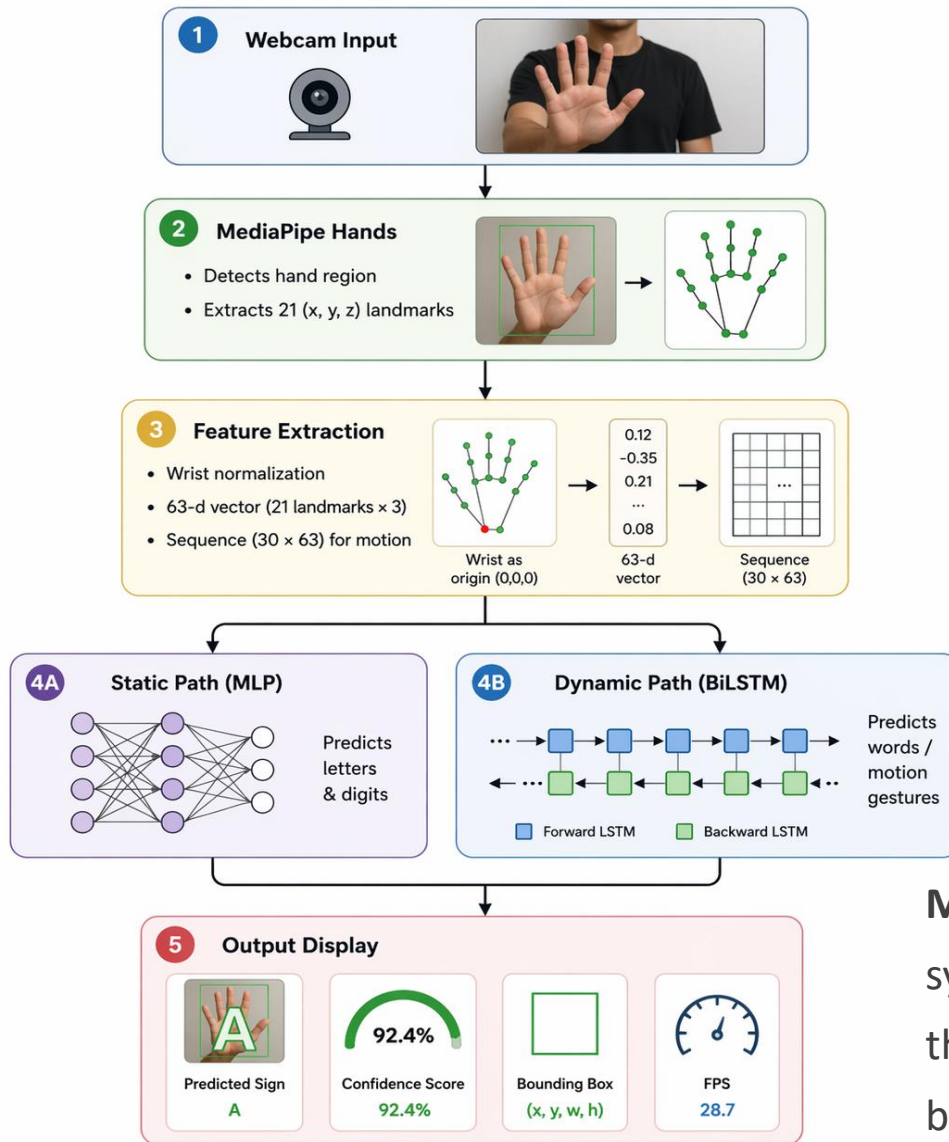
1. How effectively can a lightweight landmark-based pipeline achieve real-time ASL recognition without compromising classification accuracy?
2. What is the optimal balance between model complexity and performance in hybrid static–dynamic gesture recognition systems?
3. What are the potential advantages of using MediaPipe based hand landmark detection over the conventional CNN based pixel-wise approach regarding computational efficiency and deployment feasibility?

Existing Approaches

Approach	Description	Limitations
Image-based CNN	Classifies raw images of hand gestures with convolutional networks	Sensitive to background, lighting and skin tone; compute cost is high.
Pixel-difference techniques	Optical flow or background subtraction motion	Uses controlled environments, can be noisy.
Depth sensors (Kinect, LiDAR)	3D hand shape depth camera captured	High priced; non-portable software.
Glove-based sensors	Flex sensors on a physical glove: detect the angle of joints.	Requires users to wear hardware
ML with landmarks	Identifies hand skeleton landmarks with MediaPipe and passes them to lightweight classifiers	Chosen methodology - lightweight, privacy-aware, real-time

Why landmark-based? It is hardware-agnostic (just a webcam is required), privacy-preserving (no raw pixels are stored), resistant to changes in the background, and much lighter than CNN methods.

Proposed Framework



Mode switching: A motion heuristic is used in the system, where the dynamic mode is switched to the static mode in case the hand is found to move between successive frames.

Models Architecture & Training

Static Model — MLP

Layer	Size	Activation
Input	63	—
Hidden 1	128	ReLU
Hidden 2	64	ReLU
Hidden 3	32	ReLU
Output	34	Softmax

Dynamic Model — BiLSTM

Layer	Units	Notes
Input	(30, 63)	30-frame sequence
BiLSTM 1	64	return_sequences = True
BiLSTM 2	32	—
Dense	64	ReLU
Output	12	Softmax

Training

- Adam optimizer (lr = 0.001)
- Epochs: 50 | Batch: 32
- Validation: 15%

MLP vs BiLSTM: Key Idea

Feature	MLP	BiLSTM
Input	Single frame	30-Sequence (30 frames)
Purpose	Static gestures	Dynamic gestures
Strength	Fast, simple	Captures motion
Limitation	No time info	Slightly slower

MLP observes a single snapshot, BiLSTM grasps time change.

Key Features of the proposed approach:

1. Dual-model architecture - supports both static and motion-based gestures in one system.
2. Real-time execution - full pipeline at 37 FPS on typical CPU laptop.
3. Privacy-evasive - only 63-dimensional landmark arrays stored; no video or images stored.
4. Hardware - only a built-in webcam, no specialist equipment is required.
5. Confidence scoring - every prediction will have a probability score; a score less than 40% will be suppressed.
6. Automatic mode changing - system detects movement and changes between stationary/moving classifiers on the fly.

Data Selection:

How data was collected:

- Custom Python scripts with MediaPipe to record hand landmarks at each frame/sequence and save them.
- Recorded by a single person in a natural indoor setting with a laptop webcam.

Static Dataset

Category	Classes	Samples per Class	Total
Letters A–Y (excl. J, Z)	24	90–100	~2,280
Digits 0–9	10	100	1,000
Total	34	—	3,300

Dynamic Dataset

Category	Classes	Sequences per Class	Total
Motion letters J, Z	2	100	200
Common words	10	100	1,000
Total	12	—	1,200

Implementation and Testing

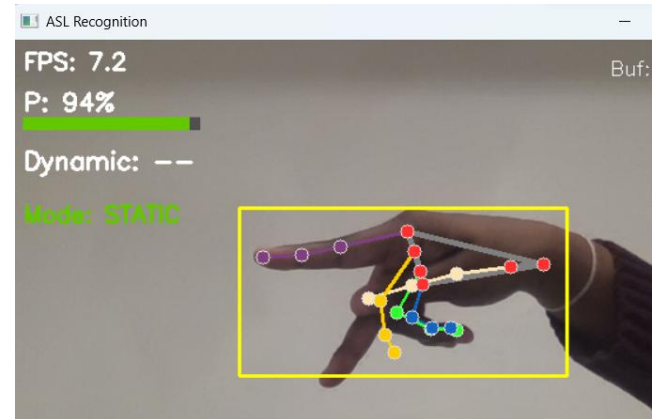
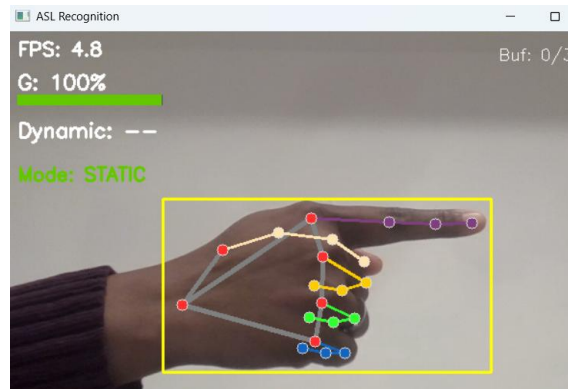
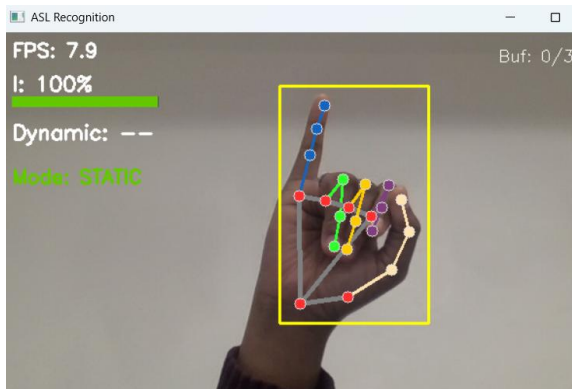
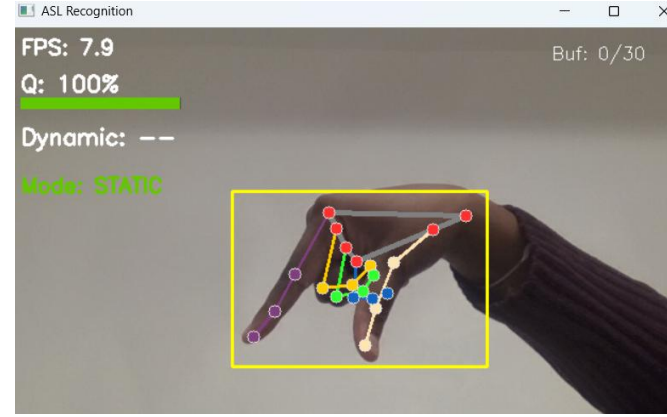
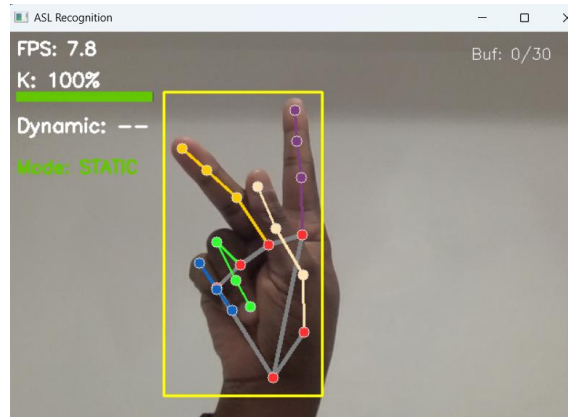
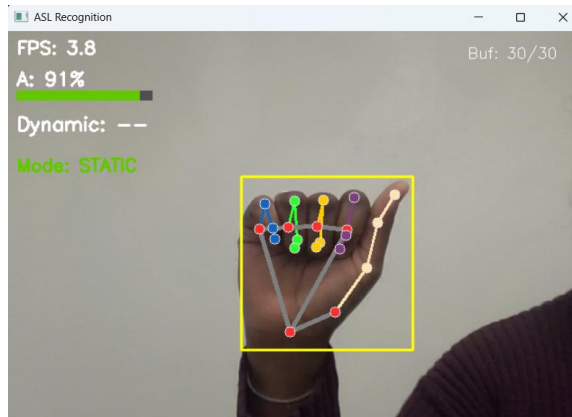
Implementation:

- Python + TensorFlow / Keras
- MediaPipe (hand tracking)
- OpenCV (webcam interface)
- NumPy, Pandas, Streamlit

Testing

- Unit testing (preprocessing pipeline)
- Model evaluation (test set + metrics)
- Performance benchmark (latency & FPS)
- Live real-time testing

Examples of gesture recognition



Results: confusion metrics

Static Model (MLP) — 34 classes

Metric	Value
Accuracy	97.37%
Weighted Precision	97.69%
Weighted Recall	97.37%
Weighted F1	97.34%
Macro F1	97.29%

Dynamic Model (BiLSTM) — 12 classes

Metric	Value
Accuracy	97.22%
Weighted Precision	97.35%
Weighted Recall	97.22%
Weighted F1	97.22%
Macro F1	97.22%

Most confused pairs (static):

- U → R (4 errors) — similar finger extension
- 0 → O (4 errors) — identical round shape
- 9 → F (2 errors) — similar loop shape

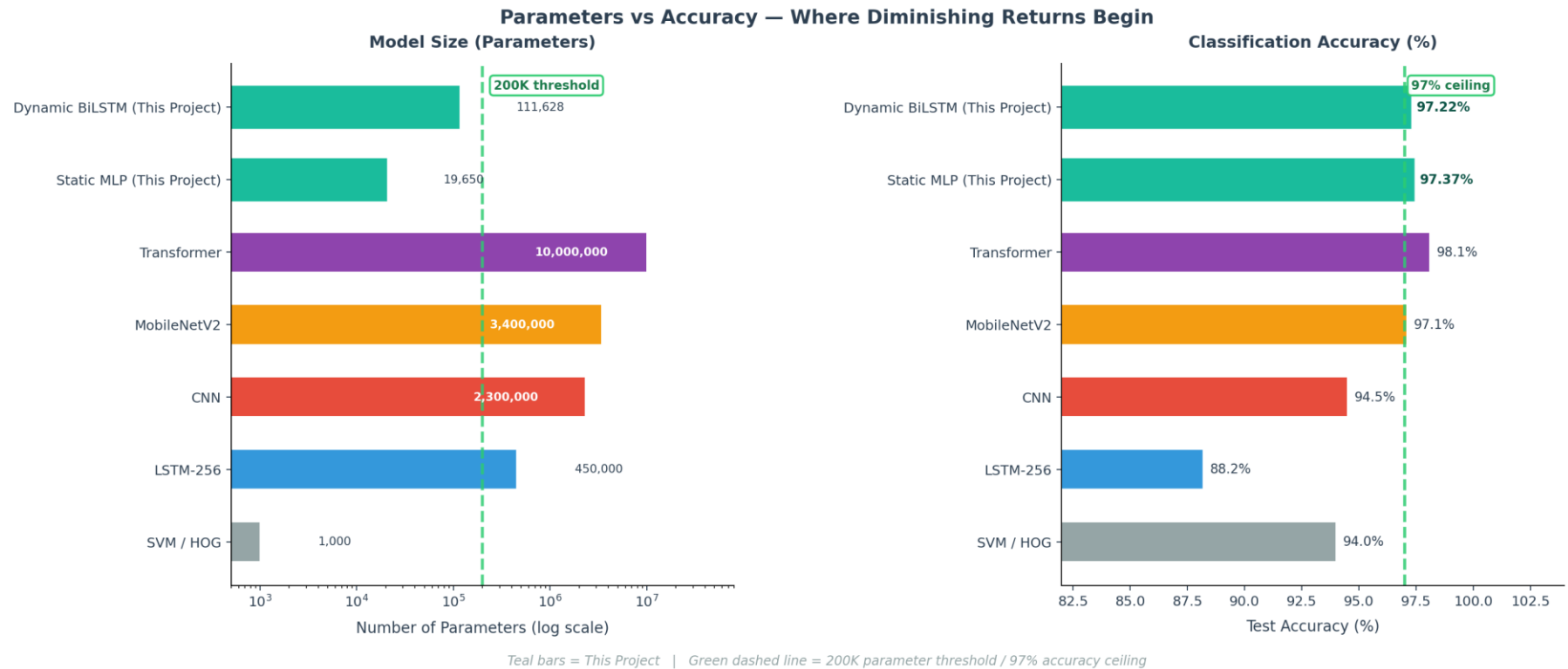
Most confused pairs (dynamic):

- J → finish (2 errors) — curved wrist motion overlap

Effectiveness comparison (RQ1)

System	Accuracy	CPU FPS	Hardware	Real-time
SVM/HOG (Cheok et al., 2019)	94.0	28+	CPU	Yes, but low accuracy
LSTM-256 (De Smedt et al., 2016)	88.2	4	CPU/GPU	Marginal
CNN (Bantupalli et al., 2018)	94.5	<3	GPU required	No
MobileNetV2 (Köpüklü et al., 2019)	97.1	<3	GPU required	No
Transformer (De Coster et al., 2021)	98.1	<2	GPU required	No
This Project- Static MLP	97.37	10.6	CPU only	Yes
This Project- Dynamic BiLSTM	97.22	10.3	CPU only	Yes

Complexity VS. Performance(RQ2)

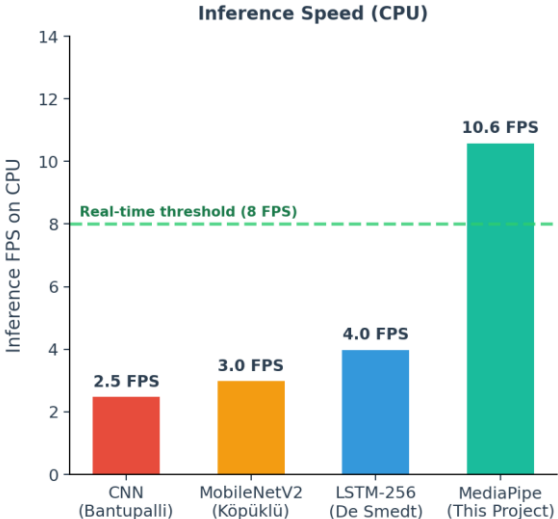


MediaPipe vs CNN(RQ3)

Metric	MediaPipe Landmark Approach	CNN Pixel-Wise Approach
Detection / Extraction FPS	37.7 FPS (26.5 ms)	5-15 FPS (GPU)
Full Pipeline FPS (CPU)	10+ FPS	< 3 FPS
Total Model Parameters	131,278	2,300,000 - 10,000,000
Model Storage Size	1.64 MB	10 MB - 200 MB
Hardware Required	CPU only	GPU for real-time
Classification Accuracy	97.37% / 97.22%	88-98%
Training Data Needed	3,300 + 1,200 samples	10,000 - 100,000+ images
Deployment Complexity	pip install mediapipe	CUDA + GPU driver + large model
Illumination Sensitivity	Low (landmark-based)	High (pixel-dependent)

FPS vs Accuracy Experiment

FPS	Dynamic Accuracy (%)	Static Accuracy (%)	Observation
5	96.67	97.37	Minimal temporal resolution, gesture severely stretched
10	97.22	97.37	Low FPS, sparse temporal sampling
15	97.22	97.37	Low FPS, sparse temporal sampling
20	97.22	97.37	Low FPS, sparse temporal sampling
30	97.22	97.37	Baseline, trained FPS
50	93.33	97.37	High FPS, incomplete gesture in buffer
60	92.78	97.37	High FPS, incomplete gesture in buffer
100	92.78	97.37	High FPS, incomplete gesture in buffer



Achieved benefits:

- Available to all with a laptop.
- Low latency (approximately 124 ms end-to-end) that can match natural communication speed.
- Easily extensible: new gestures can be discovered by gathering samples and retraining.
- Lightweight models (static: 275 KB, dynamic: 1.4 MB) can be deployed on edge devices.

Conclusions

1. The system manages to show that it is possible to implement real-time recognition of ASL on commodity hardware with a lightweight landmark-based pipeline. (RQ1)
2. The two models are above the 95% accuracy target:
 - Static MLP: 97.37% on 34 classes.
 - Dynamic BiLSTM: 97.22% in 12 classes. (RQ1)
3. The system is ethically deployable due to the privacy-preserving design (no raw video, landmarks only). (RQ2)
4. The two-model framework, which transmits pose features of the static sign and temporal sequences of the dynamic ones, turned out to be more effective than the training of a large model that can perform both. (RQ2)
5. The system achieves detection and classification speeds of about 37 and 10 FPS, which is sufficient to be used in real-time interactive. (RQ3)
6. The work creates a reproducible, open-source base that may be elaborated upon in the future by other researchers to become a complete sentence-level ASL translator. (RQ3)

Suggestions for Improvements

1. Complete ASL letter with context: Fingerspelling implies smooth motions between letters.
2. Two-hand gestures: The system is tracking one hand. ASL has numerous signs that are performed with both hands.
3. Sentence-level recognition: Integrate both static and dynamic output with a language model to create well-constructed sentences based on a sequence of gestures.
4. Mobile deployment: Transform the models to the TensorFlow Lite (TFLite) format to be deployed on Android /iOS.
5. Text output: predicting signs to a text display or text to speech engine to allow a non-signer to read or hear the output.
6. Better vocabulary: Current system includes 34 stationary + 12 dynamic classes. It would need a bigger dataset and potentially model to expand to the entire ASL vocabulary.

THANK YOU FOR ATTENTION!

Tresa Joby Thommasseri Antony

tresajoby18@gmail.com

Demo Video: <https://youtu.be/dS7H16iWKiM>

Reviewer Q1

What specific preprocessing or training modifications would you hypothesise would most effectively reduce the cross-signer accuracy gap?

The cross-signers gap occurs because all the training, validation and test data were from single signer. Wrist-relative normalization already eliminates position and scale, but it does not eliminate variability in finger length ratios, joint flexibility or signing speed.

would suggest three changes likely to make a difference:

- **Multi-signer data collection:** If we only collect 10-20 samples per gesture class from 5-10 more signers with various hand shapes and skin colors, the model will never experience this natural variance during training. The decision boundary between the two confusion pairs U/R and O/O is very close to the distribution of one signer's data and if other signers are added then the boundaries would be moved further apart.
- **Landmark-space data augmentation:** randomly disturbing training vectors during each epoch by adding small Gaussian noise to the 63 landmark coordinates and adding small random rotations to the hand plane (without collecting data), simulates the variation between signers. This is very easy to compute and can be done with little change to the model architecture itself.
- **Scale normalisation beyond wrist-relative:** if all the offsets from the landmarks were divided by the maximum offset from wrist to fingers, it wouldn't depend on the absolute size of the hand which varies between signers, yet it is a good idea, as is the case with the wrist-relative offsets. This is a single preprocessing change that may directly decrease errors from the size-sensitive ambiguous pairs such as 9 vs F.

Reviewer Q2

The frame rate sensitivity experiment shows dynamic accuracy degrades from 97.22% at 30 FPS to 92.78% at 60 FPS due to buffer truncation. Propose alternative buffer management strategies that would maintain accuracy at higher frame rates.

The root cause is that the buffer stores a fixed number of 30 frames for the gesture, independent of the capture speed, so at 60 FPS only 0.5 s of the gesture is stored. The entire 1s of gesture the model was trained with is not captured. suggesting three solutions:

- **Fixed time window:** change the 30-frame window to a 1 second window. The system receives as many frames as a camera runs at every second and reduces the number of frames or expands a variable length frame sequence to an exact 30 frames before passing it to the BiLSTM. This yields no change at 30 FPS. At 60 FPS it produces 60 frames down sampled to 30, with a full gesture duration. This would not require retraining and would probably be regained at higher frame rates (97.22%).
- **FPS-aware adaptive subsampling at inference:** detect the actual camera frame rate at start, and apply the subsampling strategy shown in the experiment to ensure that the same temporal coverage as 30 FPS training is obtained, even when the camera frame rate changes during runtime. This is very little code change, and the results of this experiment demonstrate that this method maintains accuracy well at 10–20 FPS.
- **FPS-augmented retraining:** retrain the BiLSTM on sequences that are deliberately subsampled at temporal density variations (10, 15, 20, 30 FPS), which makes the model itself robust to temporal density variation. This is the most complete solution and will take retraining. If used with option 1 it will allow the system to be deployed on any piece of hardware, irrespective of camera speed.

Reviewer Q3

Landmark data is derived from video and could theoretically be reverse-engineered. What additional privacy-preserving techniques could be applied to your pipeline?

The thesis already has an initial layer of privacy in that it only saves 63 landmark coordinates, and never raw frames. However, I acknowledge it is possible in theory to attempt to reconstruct an approximate hand geometry from precise coordinates. I'd suggest three complementary ways:

- **Feature transformation to derived representations:** represent x , y , z vectors of displacement as inter-landmark distances and joint angles instead of storing raw (x, y, z) vectors. These derived features carry all the information necessary for the classifier, but they are not easily converted to exact coordinates. This is a purely pre-processing change with no computational overhead and eliminates spatial position reconstruction capability.
- **Coordinate perturbation:** Add calibrated Laplacian/gaussian noise to the landmark coordinates before storing/transmitting them. The size of the noise magnitude can be tweaked to not affect classification accuracy (my FPS experiment showed that the model was robust enough to not be affected by small temporal perturbations; the same seems to be true for small spatial noise) but reverse engineering becomes mathematically impractical. This is the most established formal privacy guarantee.
- **Future multi-signer training using on-device inference with federated learning:** the current system is already running fully on-device, so there is no risk of data being transmitted. With the system extended to learn from multiple users to close the cross-signer gap (Question 1), federated learning would enable each system to train locally and only send gradient updates to the system. The model evolves collaboratively without any raw data being shared between the user's device. This solves the problem of storing landmark data and at the same time provides the necessary diversity of signers for the system.

Of these three, the feature transformation to angles and distances is the one I would prioritise as an immediate implementation step, since it eliminates the reverse-engineering concern entirely without any trade-off in accuracy or computational cost.